

DMA传输

DMA传输

一、学习目标

DMA原理

二、硬件搭建

三、实验步骤

1. 打开SYSCONFIG配置工具

2. ADC参数配置

3. DMA参数配置

4. 写入程序

5. 编译

四、程序分析

五、实验现象

一、学习目标

1.了解DMA基本知识。

2.学习DMA的基本使用，将传输的ADC数据显示在串口助手上。

DMA原理

DMA（Direct Memory Access，直接存储器访问）是一种用于计算机系统的技术，使外设可以直接访问内存，而无需通过CPU来完成数据的传输。它使得数据在外设（如ADC、DAC、存储器等）和内存之间的交换能够高效地进行，减少了CPU的负担，提高了系统的整体性能。

原理：

- **触发数据传输：**DMA操作通常由外部设备（如ADC、传感器）或内部事件（如定时器溢出）触发。
- **初始化DMA控制器：**CPU向DMA控制器配置源地址、目标地址、传输数据大小、传输方向等参数。
- **直接传输数据：**一旦DMA控制器被激活，它能够直接从源地址（外设的输出缓冲区或内存）将数据传输到目标地址（内存或外设），而无需CPU干预。
- **传输完成后通知CPU：**DMA完成数据传输后，可以向CPU发送中断请求，告诉CPU数据已经准备好，或者传输已成功完成。

二、硬件搭建

MSPM0G3507的DMA控制器具有以下特点：

- 7个独立的传输通道；
- 可以配置的DMA通道优先级；
- 支持8位(byte)，16位(short word)、32位(word)和64位(long word)或者混合大小(byte 和 word)传输；
- 支持最大可达64K任意数据类型的数据块传输；

- 可配置的DMA传输触发源；
- 6种灵活的寻址模式；
- 单次或者块传输模式： 它共有7个DMA通道，各个通道可以独立配置，多种多样的数据传输模式可以适应不同应用场景的数据传输需要。

通过查看TI的数据手册，DMA功能除了常见的内存与外设间的地址寻址方式，还提供了Fill Mode和Table Mode两种拓展模式，DMA通道分为基本类型和全功能类型两种。

Table 4-1. Feature Comparison of Basic and Full-Feature DMA Channels

DMA Feature	Full-Feature Channel	Basic Channel
Repeated mode	✓	–
Early IRQ notification	✓	–
Block burst mode	✓	✓
Stride mode	✓	✓
Internal channel as trigger source	✓	✓
Extended Mode (Table and Fill Mode)	✓	–

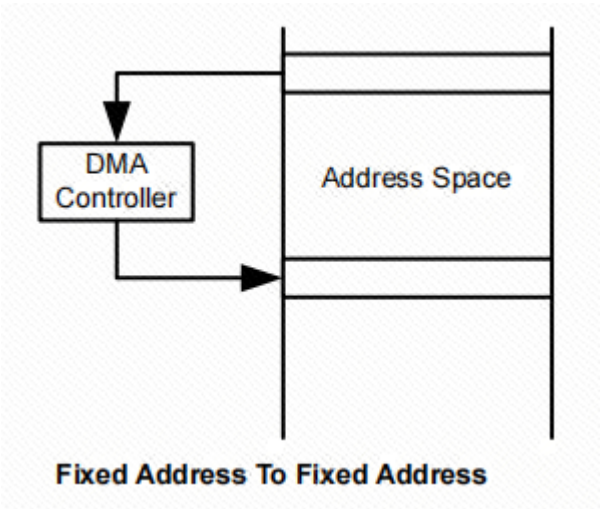
通过查看数据手册，只有全功能类型的DMA通道支持重复传输、提前中断以及拓展模式，基本功能的DMA通道支持基本的数据传输和中断，但是足够满足简单的数据传输要求。

MSPM0G3507的DMA支持四种传输模式，每个通道可单独配置其传输模式。例如，通道 0 可配置为单字或单字节传输模式，而通道 1 配置为块传输模式，通道 2 采用重复块传输模式。传输模式独立于寻址模式进行配置。任何寻址模式都可以与任何传输模式一同使用。可以传输三种类型的数据，可以通过 .srcWidth 和 .destWidth 控制位进行选择。源位置和目标位置可以是字节、短字或字数据。也支持以字节到字节、短字到短字、字到字或任何组合的方式进行传输。如下：

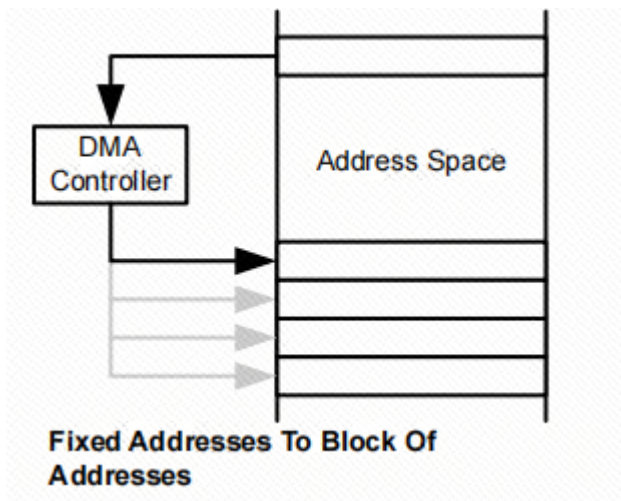
- 单字或单字节传输：每次传输都需要一个单独的触发。当 DMAxSZ 传输已经生成时 DMA 会被自动关闭。
- 块传输：一个整块将会在一个触发后传输。在块传输结束时 DMA 会被自动关闭。
- 重复单字或单字节传输：每次传输都需要一个单独的触发，DMA保持启用状态。
- 重复块传输：一个整块将会在一个触发后传输，DMA保持启用状态。

基于这三种模式，MSPM0G3507提供了6种寻址方式，

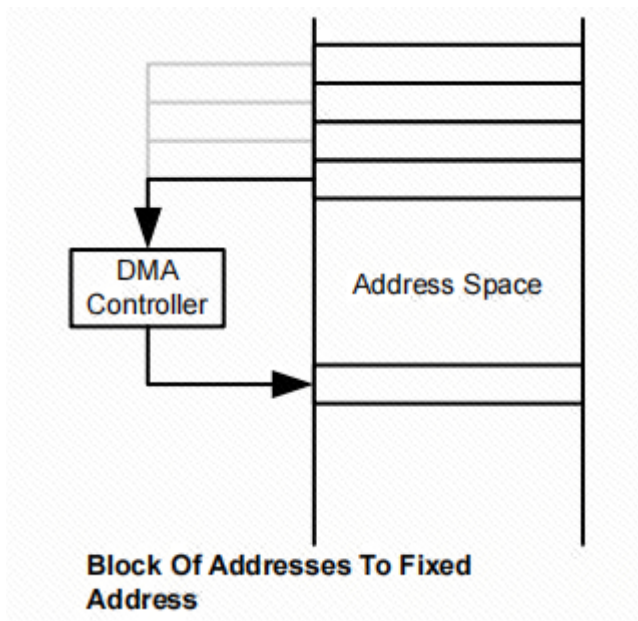
固定地址到固定地址：



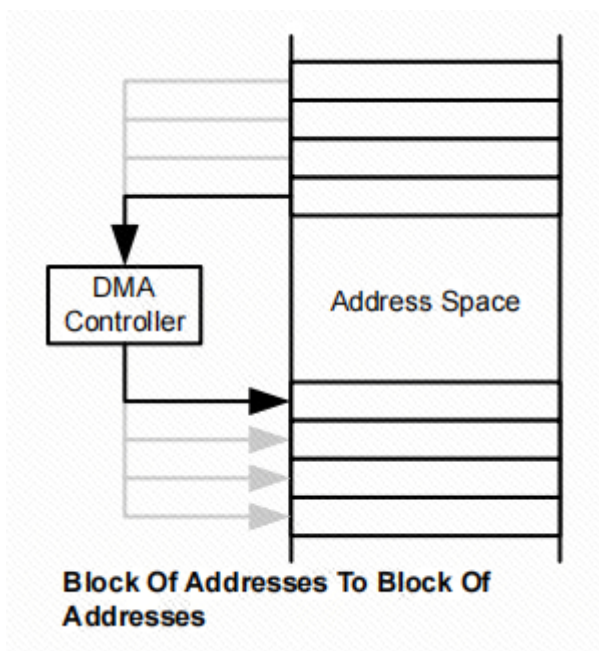
固定地址到地址块：



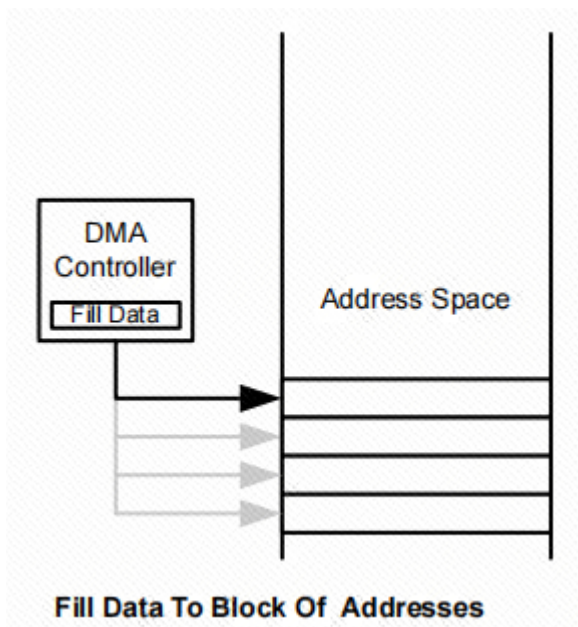
地址块到固定地址：



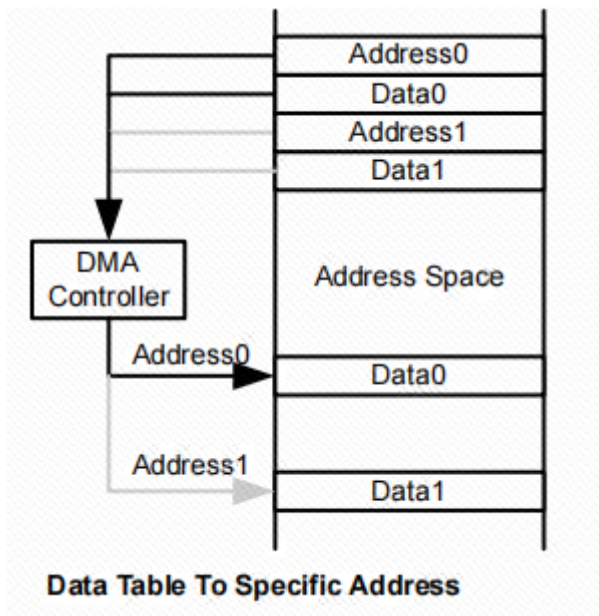
地址块到地址块：



将数据填充到地址块：



数据表到指定地址：

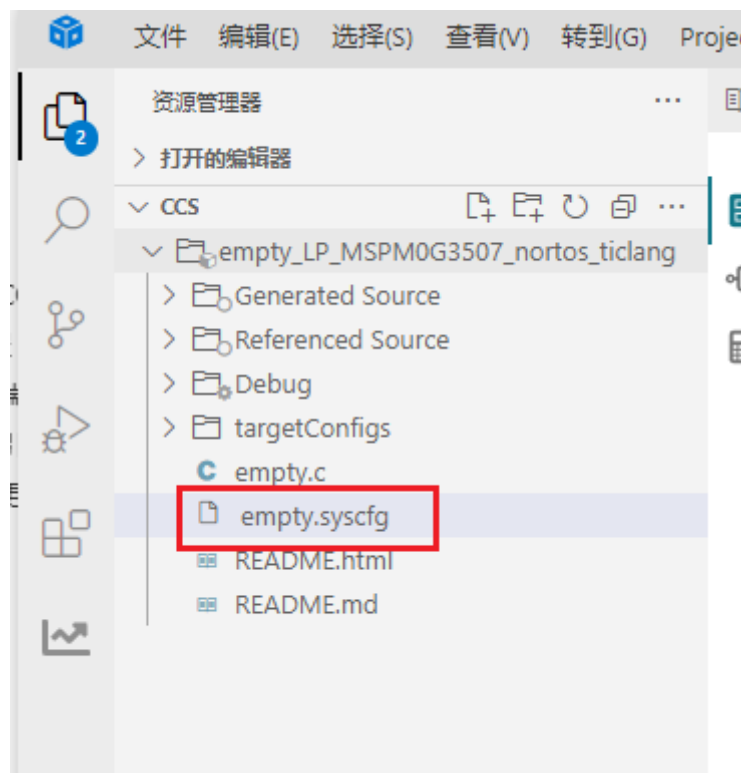


三、实验步骤

本案例将使用ADC采集PA27引脚的电压，通过DMA将ADC采集的数据直接搬运到指定地址中。

1. 打开SYSCONFIG配置工具

这里使用 **ADC采集** 课程为模板进行介绍。打开ADC工程，并打开.syscfg文件。



2. ADC参数配置

这里ADC的配置与**ADC**采集案例相同。

配置ADC的分辨率和中断。本案例不使用中断，这里什么都不选。

Advanced Configuration

Total Conversion Frequency	9.78 kHz When Power Down Mode is set to Auto, ADC wakeup time may need to be considered in each sample window. Refer to the device specific data sheet for specifications on the ADC Wakeup Time.
Conversion Resolution	12-bits
Enable FIFO Mode	☐
Power Down Mode	Auto
Desired Sample Time 0	100 us
Actual Sample Time 0	100.00 μ s
Desired Sample Time 1	0 ms
Actual Sample Time 1	0.00 s

Interrupt Configuration

Enable Interrupts	None
-------------------	------

3. DMA参数配置

DMA Configuration

Configure DMA Trigger

☒

Enable DMA Trigger

☒

DMA Samples Count

1

Enable DMA Triggers

MEM0 result loaded interrupt

DMA Channel

Name

DMA_CH0

Channel ID

0

Currently using a Full Channel.

Address Mode

Fixed addr. to Block addr.

Source Length

Half Word (2 Bytes)

Destination Length

Half Word (2 Bytes)

Destination Address Direction

Increment

Configure Transfer Size

☒

Transfer Size

10

Transfer Mode

Repeat Single

Source Address Increment

Do not change address after each transfer

Destination Address Increment

Do not change address after each transfer

Interrupt Configuration

Enable Channel Interrupt

☐

Enable Early Channel Interrupt

☐

通过快捷键 Ctrl+S 将.syscfg文件中的配置保存。

4. 写入程序

在empty.c文件中，写入以下代码

```
#include "ti_msp_dl_config.h"
#include "stdio.h"

volatile uint16_t ADC_VALUE[20]; //ADC采集的数据保存地址 The data collected by ADC is saved in the address

unsigned int adc_getValue(unsigned int number); //读取ADC的数据 Read ADC data
void uart0_send_string(char* str); //串口发送字符串 Send string via serial port

int main(void)
{
    char output_buff[50] = {0};
    unsigned int adc_value = 0;
    float voltage_value = 0;

    SYSCFG_DL_init();

    //设置DMA搬运的起始地址 Set the starting address of DMA transfer
    DL_DMA_setSrcAddr(DMA, DMA_CH0_CHAN_ID, (uint32_t) &ADC0->ULLMEM.MEMRES[0]);
    //设置DMA搬运的目的地址 Set the destination address of DMA transfer
```

```

DL_DMA_setDestAddr(DMA, DMA_CH0_CHAN_ID, (uint32_t) &ADC_VALUE[0]);
//开启DMA Enable DMA
DL_DMA_enableChannel(DMA, DMA_CH0_CHAN_ID);
//开启ADC转换 Start ADC conversion
DL_ADC12_startConversion(ADC_VOLTAGE_INST);

uart0_send_string("adc+dma Demo start\r\n");
while (1)
{
    //获取ADC数据 Get ADC data
    adc_value = adc_getValue(10);
    sprintf(output_buff, "adc value:%d\r\n", adc_value);
    uart0_send_string(output_buff);

    //将ADC采集的数据换算为电压 Convert the data collected by ADC into voltage
    voltage_value = adc_value/4095.0*3.3;
    sprintf(output_buff, "voltage value:%.2f\r\n", voltage_value);
    uart0_send_string(output_buff);

    delay_cycles(32000000);
}
}

//读取ADC的数据 Read ADC data
unsigned int adc_getValue(unsigned int number)
{
    unsigned int gAdcResult = 0;
    unsigned char i = 0;

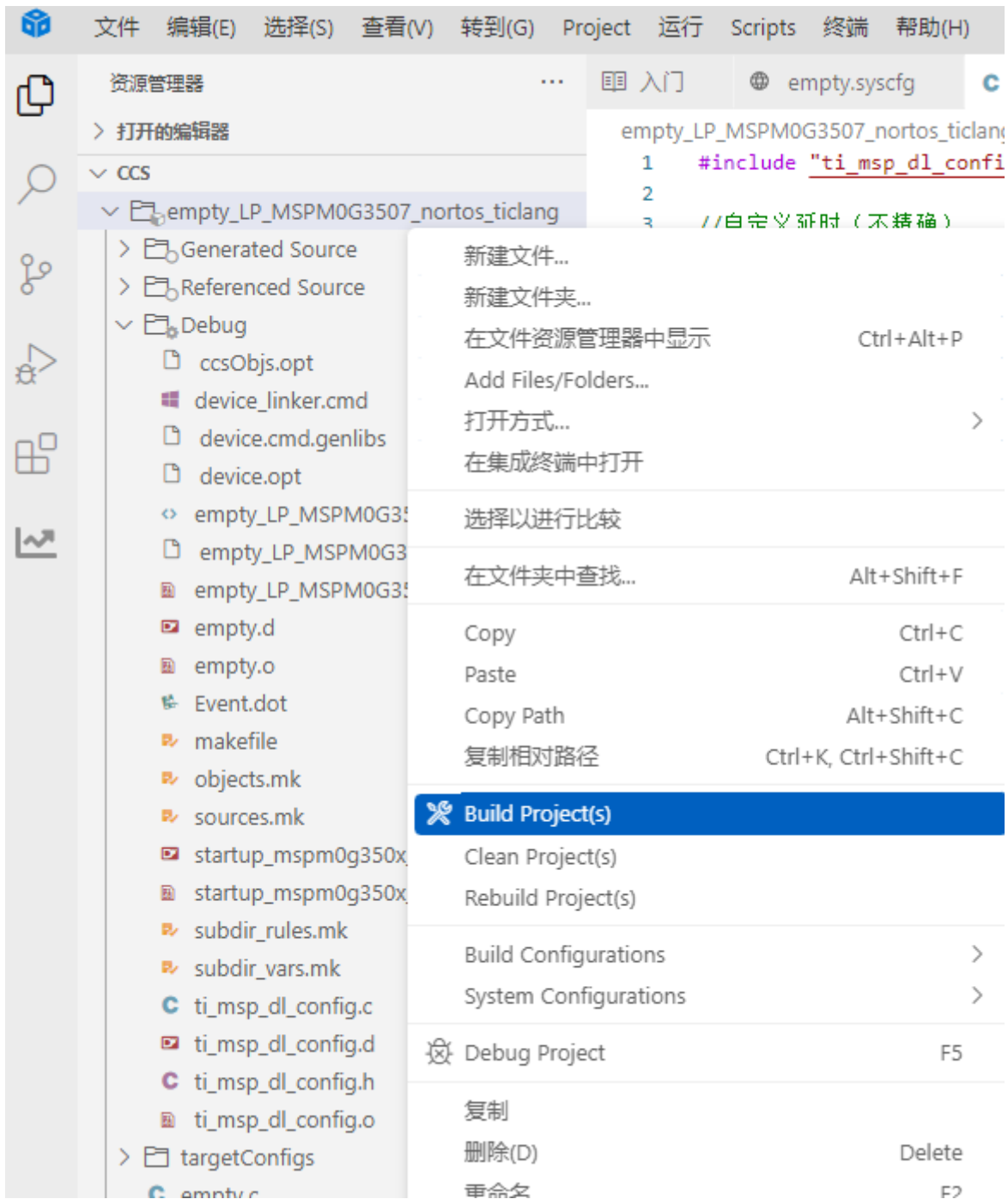
    //采集多次累加 Collect multiple times and accumulate
    for( i = 0; i < number; i++ )
    {
        gAdcResult += ADC_VALUE[i];
    }
    //均值滤波 Mean Filter
    gAdcResult /= number;

    return gAdcResult;
}

//串口发送字符串 Send string via serial port
void uart0_send_string(char* str)
{
    //当前字符串地址不在结尾 并且 字符串首地址不为空
    //The current string address is not at the end and the string first address
    is not empty
    while(*str!=0&&str!=0)
    {
        //当串口0忙的时候等待, 不忙的时候再发送传进来的字符
        // wait when serial port 0 is busy, and send the incoming characters
        when it is not busy
        while( DL_UART_isBusy(UART_0_INST) == true );
        //发送字符串首地址中的字符, 并且在发送完成之后首地址自增
        // Send the characters in the first address of the string, and increment
        the first address after sending.
        DL_UART_Main_transmitData(UART_0_INST, *str++);
    }
}

```

5. 编译



编译成功了，即可将程序下载到开发板。

四、程序分析

- empty.c


```

60 //串口发送字符串 Send string via serial port
61 void uart0_send_string(char* str)
62 {
63     //当前字符串地址不在结尾 并且 字符串首地址不为空
64     //The current string address is not at the end and the string first address is not empty
65     while(*str!=0&&str!=0)
66     {
67         //当串口0忙的时候等待, 不忙的时候再发送传进来的字符
68         // Wait when serial port 0 is busy, and send the incoming characters when it is not busy
69         while( DL_UART_isBusy(UART_0_INST) == true );
70         //发送字符串首地址中的字符, 并且在发送完成之后首地址自增
71         // Send the characters in the first address of the string, and increment the first address after sending.
72         DL_UART_Main_transmitData(UART_0_INST, *str++);
73     }
74 }

```

`uart0_send_string` 函数通过UART 串口发送一个字符串。通过 `while (*str != 0)` 循环遍历每个字符, 直到字符串结束 (`*str == 0`), 在发送每个字符之前, 通过 `DL_UART_isBusy(UART_0_INST)` 检查 UART 是否正在忙。只有当 UART 空闲时, 才能发送数据。通过 `DL_UART_Main_transmitData(UART_0_INST, *str++)` 将当前字符发送出去, 并让 `str` 指向下一个字符。

```

43 //读取ADC的数据 Read ADC data
44 unsigned int adc_getValue(unsigned int number)
45 {
46     unsigned int gAdcResult = 0;
47     unsigned char i = 0;
48
49     //采集多次累加 Collect multiple times and accumulate
50     for( i = 0; i < number; i++ )
51     {
52         gAdcResult += ADC_VALUE[i];
53     }
54     //均值滤波 Mean Filter
55     gAdcResult /= number;
56
57     return gAdcResult;
58 }

```

这段代码读取 ADC (模拟数字转换器) 数据。通过触发 ADC 转换并等待转换完成, 最后获取 ADC 采样结果并返回。

```

9   int main(void)
10  {
11      char output_buff[50] = {0};
12      unsigned int adc_value = 0;
13      float voltage_value = 0;
14
15      SYSCFG_DL_init();
16
17      //设置DMA搬运的起始地址 Set the starting address of DMA transfer
18      DL_DMA_setSrcAddr(DMA, DMA_CH0_CHAN_ID, (uint32_t) &ADC0->ULLMEM.MEMRES[0]);
19      //设置DMA搬运的目的地址 Set the destination address of DMA transfer
20      DL_DMA_setDestAddr(DMA, DMA_CH0_CHAN_ID, (uint32_t) &ADC_VALUE[0]);
21      //开启DMA Enable DMA
22      DL_DMA_enableChannel(DMA, DMA_CH0_CHAN_ID);
23      //开启ADC转换 Start ADC conversion
24      DL_ADC12_startConversion(ADC_VOLTAGE_INST);
25
26      uart0_send_string("adc+dma Demo start\r\n");
27      while (1)
28      {
29          //获取ADC数据 Get ADC data
30          adc_value = adc_getValue(10);
31          sprintf(output_buff, "adc value:%d\r\n", adc_value);
32          uart0_send_string(output_buff);
33
34          //将ADC采集的数据换算为电压 Convert the data collected by ADC into voltage
35          voltage_value = adc_value/4095.0*3.3;
36          sprintf(output_buff, "voltage value:%.2f\r\n", voltage_value);
37          uart0_send_string(output_buff);
38
39          delay_cycles(32000000);
40      }
41  }

```

这段代码的功能是通过 **ADC** (模数转换器) 和 **DMA** (直接内存存取) 结合，进行模拟信号的采集并将结果通过串口输出。通过 DMA 将 ADC 数据直接搬运到内存，不需要每次手动读取，并定期通过串口输出 ADC 采样值和计算的电压值。

五、实验现象

程序下载完成之后，对串口助手进行如下图配置之后，调节电位器旋钮，向PA27引脚输出电压，ADC将数据采集装载到ADC结果寄存器0后，DMA被触发，DMA将会把数据搬运到内存变量ADC_VALUE中，通过直接调用ADC_VALUE即可得到数据。

